Soup DevOps release pipeline

Maksim | 10 November 2026 | Evidence and verdict

A green source test is insufficient: the distribution must work after installation. This repository builds a dependency-free greeting CLI, deliberately breaks four independent release properties, and validates the published candidate on Windows and Ubuntu, CPython 3.10-3.14. Private repository.

Four controlled incidents

Python compatibility. Import datetime.UTC (introduced in 3.11) while advertising 3.10 support. Both 3.10 artifact jobs fail with ImportError; the other eight combinations pass. Naive: GREEN / Hardened: RED.

OS portability. Replace getpass with the Unix-only pwd module. All five Windows artifact jobs fail with ModuleNotFoundError; Linux passes. Naive: GREEN / Hardened: RED.

Packaging completeness. Exclude greeting.txt from package data. Tests in src still see it; installed wheel and rebuilt sdist raise FileNotFoundError on all ten targets, and Docker fails. Naive: GREEN / Hardened: RED.

Console entry point. Point the script at missing_main. Calling main() in unit tests still works; the installed command raises ImportError on all ten targets, and Docker fails. Naive: GREEN / Hardened: RED.

The injector prints an exact diff against a clean checkout; the same commit is used for each pair. Naive CI tests source on Linux 3.12, builds, and checks metadata. The hardened path exercises the installed console command, not just importability. All original failed runs remain available in docs/evidence.md.

What the corrected pipeline verifies

Lint and unit tests precede one wheel/sdist build using hash-locked tools. The wheel is built from the sdist. Ten OS/Python jobs install the wheel and independently rebuild the sdist in a builder, then test it in another clean runtime. Runtimes exclude the checkout and bootstrap packages. Docker installs the same wheel and runs non-root, offline and read-only. Corrected matrix: GREEN.

On a matching tag reachable from main, only the publisher receives OIDC permission in the tag-restricted testpypi environment. It uploads the tested bytes without rebuilding. A final job checks both file hashes served by TestPyPI and performs a fresh, hash-checked index installation. Release and index check: GREEN; TestPyPI 0.9.3.

Trust boundaries, costs and limitations

Silent failures addressed

Dependencies: no runtime dependencies; unexpected Requires-Dist is rejected. Build tools and transitive requirements use exact versions and hashes; Actions and the Docker base use immutable references. These choices reduce moving inputs, but hashes cannot establish that a selected dependency is benign.

Cache and stale artifacts: no environment or resolver cache is restored. Fresh runtime venvs cannot borrow build dependencies. Commit, run ID, attempt, version and both distribution hashes are bound in a manifest. Every consumer verifies the same immutable upload; negative checks reject tampering, a stale commit and a wrong tag.

Privilege and version: PR jobs have read-only permissions, no publisher secrets, no persisted checkout credentials, and no privileged PR event. OIDC exists only in the tag-only publish job. A version/tag mismatch fails before upload. These controls target high-impact failures without buying GPU time or adding runtime dependencies.

GPU design and monthly estimate

GPU testing is a design only: this CPU-only CLI has no useful GPU behavior to validate. For a future GPU feature, use a Runpod Secure Cloud RTX 4090 at $0.89/hour, checked 10 November 2026. Assume 30 nightlies at 10 minutes, four releases at 20 minutes, and two minutes overhead per run: 7.47 hours, $6.65 compute. Prorated disposable 20 GB disk adds about $0.02; a 25% allowance gives **$8.33/month**. Excludes tax, egress and GitHub minutes. Provider prices.

Run only after CPU gates, on isolated ephemeral workers. A broker validates GitHub OIDC and keeps the provider key out of workloads. Reserve worst-case lease cost, cap concurrency and hourly price, enforce a $10 monthly cap, and use an external watchdog to terminate Pods and disks at 15/30-minute limits even after Actions cancellation. No paid GPU was rented.

Verdict: not ready for unattended real PyPI

Automatic TestPyPI publication is demonstrated. Before real PyPI, protect main and release tags, require review of workflow/lock changes, and add tests for the actual application's contracts. Green does not prove absence of malicious code, vulnerable pinned tools or account compromise. macOS, ARM, PyPy and unusual user environments are untested. Post-publication verification can detect failure only after upload. Yank a bad release and publish a new version; this cannot recall existing installations or mirrored files.

Surprise and remaining concern

The strongest surprise was that valid metadata, a successful build and passing source tests all coexisted with an unusable installed command. Even importing the package would miss the broken entry point. The remaining concern is the workflow's own trust boundary: a compromised maintainer can change both code and checks, then tag a malicious release.

AI usage and personal verification

Codex drafted code, workflows and prose and executed the automated checks. Review corrected YAML quoting, Docker context exclusions, hidden bootstrap dependencies, optimization-sensitive assertions and initially suppressed CLI stderr. The applicant configured the TestPyPI Trusted Publisher and independently created a fresh local venv, installed version 0.9.3 from TestPyPI, and ran the installed CLI. The observed JSON contained Hello, Soup! and version 0.9.3. The broader matrix and negative tests were automated; no manual review of every Actions log is claimed.